DEOS Ethernet Transport Adapter (TCP & UDP) ICD

Sürüm 1.1 – Textual (TCP) & ROS 2 Bridge (UDP) Arayüzleri

Emir Arkalı (emirarkali07@gmail.com)
DEOS – Dokuz Eylül Otonom Sistemler

İlk Yayın: 28 Eylül 2026 | Son Revizyon: 13 Ekim 2026

Doküman Tipi:	Interface Control Document (ICD)	**Doküman Kodu:**	`DOC-ICD-TRP-ETH-006`
Doküman Sürümü:	Sürüm 1.1	**Protokol Sürümü:**	v1.1 (`0x11`) – Medya Bağımsız
Yazar:	Emir Arkalı (`emirarkali07@gmail.com`)	**Son Değiştiren:**	Emir Arkalı
İlk Yayın Tarihi:	28 Eylül 2026	**Son Revizyon Tarihi:**	13 Ekim 2026
Durum:	Yürürlükte / Onaylandı	**Taşıma / Kapsam:**	TCP (Port 5000) & UDP (Port 5001) Taşıma Katmanı

Doküman Revizyon Geçmişi

Revizyon	Tarih	Yazar / Değiştiren	Açıklama
v0.3	28 Eylül 2026	Emir Arkalı	İlk taslak ve monolitik ICD v0.3 tanımı.
v1.0	04 Ekim 2026	Emir Arkalı	TCP/Ethernet adaptörü bağımsız doküman oluşturuldu.
v1.1	13 Ekim 2026	Emir Arkalı	Sürüm 1.1 standardizasyonu. Yerel ROS 2 UDP Bridge (Port 5001) taşıma katmanı entegre edildi; Textual ve ROS Bridge Node ID eşleşmeleri düzeltildi.

1 Giriş ve Kapsam

Bu doküman, DEOS ağındaki Araç Ana Kontrolcüsü (`DEOS_NODE_MAIN_STM32`, `0x03`) üzerinde koşan Ethernet taşıma katmanının kurallarını ve çerçeve (framing) yapısını tanımlar.

DEOS Ethernet donanım hattı, üzerindeki tek fiziksel PHY/MAC üzerinden iki farklı mantıksal soket arayüzü sunar:

1. **Textual Konsol Arayüzü (TCP Port 5000):** Operatör bilgisayarı veya Raspberry Pi üzerindeki Python/CLI arayüzü ile bağlantı odaklı, sıralı ve garantili akış haberleşmesi.
2. **ROS 2 Köprü Arayüzü (UDP Port 5001):** Gömülü Linux otonom sürüş bilgisayarı üzerindeki `deos_ros_bridge` paketi ile çift yönlü, düşük gecikmeli, datagram tabanlı haberleşme.

Ethernet bağlantısında fiziksel katman (L2) donanımsal 32-bit CRC (FCS) doğrulaması yaptığı ve L4 taşıma protokolleri (TCP/UDP) kendi dahili checksum mekanizmalarını barındırdığı için, LoRa tarafında kullanılan yazılımsal CRC16 hesaplamalarına bu hatta **ihtiyaç duyulmaz**. Bununla birlikte, TCP akışındaki olası bayt kaymalarına karşı koruma sağlamak amacıyla TCP çerçevelerinin başında `0xAA 0x55` senkronizasyon baytları yer alır; mesaj tabanlı çalışan UDP tarafında ise ek bir magic bayta gerek duyulmaz.

2 Textual TCP Konsol Arayüzü (Port 5000)

2.1 Bağlantı Özellikleri

- **Protokol:** TCP Stream (`SOCK_STREAM`)
- **Roller:** Zephyr (STM32) kartı **TCP Server** olarak dinlemede kalır. Textual arayüzü (Python) açıldığı anda **TCP Client** olarak karta bağlanır.
- **Varsayılan IP & Port:** `192.168.1.10:5000`
- **Kaynak/Hedef Düğüm:** `DEOS_NODE_TEXTUAL` (`0x01`)

2.2 TCP Çerçeve (Frame) Formatı

TCP veri akışında çerçeve sınırlarını belirlemek ve akış kaymalarına karşı anında yeniden senkronize olabilmek (Stream Resynchronization) için her paketin başına 2 baytlık `0xAA 0x55` sihirli baytları (Magic Word) ve 1 baytlık `Length` bilgisi eklenir:

Magic Word (2 Bayt)	Length (1 Bayt)	DEOS Common Header (8 Bayt)	DEOS Payload (N
`0xAA 0x55`	`0x08 - 0x44`	Priority, Class, Service, Dest, Src, Ver, Seq, Cmd	Komut Verisi (Örn:

- **Magic Word (2 Byte):** Daima `0xAA 0x55` değerindedir. Alıcı akıştan koparsa bir sonraki `0xAA 0x55` desenini arayarak akışı anında kurtarır.
- **Length (1 Byte):** Kendisinden sonra gelen DEOS Header ve Payload alanlarının toplam bayt sayısıdır (`8 + payload_len`).
- **DEOS Common Header (8 Byte):** Sırasıyla `priority`, `message_class`, `service`, `destination`, `source`, `version`, `sequence`, `command`.
- **DEOS Payload (N Byte):** Gönderilmeyen boş baytlar hatta basılmaz; yalnızca dolu olan N bayt aktarılır.

3 ROS 2 Köprüsü UDP Arayüzü (Port 5001)

3.1 Bağlantı Özellikleri

- **Protokol:** UDP Datagram (`SOCK_DGRAM`)
- **Roller:** Zephyr (STM32) kartı tüm arayüzlerde (`INADDR_ANY`) **5001** portunu dinler. Otonom kontrol tarafındaki `deos_ros_bridge` C++ node'u bu porta datagram yollar.
- **Varsayılan IP & Port:** `192.168.1.10:5001`
- **Mantıksal Düğüm:** `DEOS_NODE_ROS_BRIDGE` (`0x04`)

3.2 Dinamik İstemci Keşfi (Dynamic Client Discovery)

ROS 2 köprüsü statik bir IP'ye zorlanmak zorunda değildir. STM32 Main Node (`eth_ros_bridge.cpp`):

1. Port 5001 üzerinden gelen ilk geçerli pakette göndericinin IP adresini ve port numarasını (`m_client_addr`) hafızaya alır (*Discovered*).
2. Araç içindeki CAN düğümlerinden veya sensörlerden ROS 2 köprüsüne (`DEOS_NODE_ROS_BRIDGE`) gitmesi gereken tüm telemetri mesajlarını `zsock_sendto` ile bu keşfedilen istemci adresine basar.
3. İstemci adresi veya portu değişirse, gelen yeni paketle birlikte hedef adres otomatik güncellenir.

3.3 UDP Datagram Paket Yapısı

UDP datagramları sınırları belirli paketler (packet-oriented) halinde taşınır. Her UDP datagramının başında 1 baytlık uzunluk göstergesi yer alır:

Length (1 Bayt)	DEOS Common Header (8 Bayt)	DEOS Payload (N Bayt)
$8 + N$	8 Bayt Sabit Başlık	Dinamik Veri Baytları

- Toplam datagram boyutu: `Length + 1` bayttır.
- Alıcı taraf, gelen datagramın ilk baytını okur (`rx_len = rx_buf[0]`); eğer alınan toplam bayt sayısı `rx_len + 1` değerine eşit ve `rx_len` ≥ 8 ise paketi doğrudan DEOS çekirdeğine iletir.

4 Örnek Mesaj Anatomisi (PING Mesajı)

İstemciden (Textual veya ROS 2 Bridge) karta atılan bir PING paketinin bayt dizilimi:

Byte Indeksi	Alan	Örnek (Hex)	Açıklama
0	Length	0x0C	12 Byte (8 Bayt Header + 4 Bayt Payload)
1	priority	0x01	DEOS_PRIO_NETWORK
2	message_class	0x01	DEOS_CLASS_NETWORK
3	service	0x00	DEOS_SERVICE_SYSTEM
4	destination	0x03	DEOS_NODE_MAIN_STM32
5	source	0x04	DEOS_NODE_ROS_BRIDGE (0x01 Textual için)
6	version	0x11	Protokol Versiyonu (v1.1)
7	sequence	0x05	Mesaj Sıra Numarası
8	command	0x01	DEOS_CMD_SYSTEM_PING
9	payload[0]	0x45	PING ID: 69 (Little Endian) LSB
10	payload[1]	0x00	PING ID
11	payload[2]	0x00	PING ID
12	payload[3]	0x00	PING ID MSB

5 Referans İstemci Kodları

5.1 Python TCP İstemci (Textual - Port 5000)

```
import socket, struct

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("192.168.1.10", 5000))

# 8 bayt DEOS Header: Prio, Class, Srv, Dest(0x03), Src(0x01), Ver(0x11), Seq, Cmd(0x01)
header = struct.pack('<8B', 1, 1, 0, 0x03, 0x01, 0x11, 1, 0x01)
payload = struct.pack('<I', 69) # Ping ID: 69
msg = header + payload

magic = bytes([0xAA, 0x55])
length_byte = bytes([len(msg)]) # 12 Byte
sock.sendall(magic + length_byte + msg)
```

5.2 C++ ROS 2 Bridge UDP İstemci (Port 5001)

```
// deos_ros_bridge icerisindeki udp_transport ornegi
uint8_t tx_buf[64];
uint8_t payload_len = 4;
uint8_t len = 8 + payload_len; // 12

tx_buf[0] = len;
tx_buf[1] = DEOS_PRIO_COMMAND;
tx_buf[2] = DEOS_CLASS_COMMAND;
tx_buf[3] = DEOS_SERVICE_TRACTION;
tx_buf[4] = DEOS_NODE_TRACTION;      // Hedef: 0x10
tx_buf[5] = DEOS_NODE_ROS_BRIDGE;    // Kaynak: 0x04
tx_buf[6] = 0x11;                    // DEOS v1.1
tx_buf[7] = seq++;
tx_buf[8] = 0x01;                    // SET_SPEED

// Payload (Hedef hiz int32: 500 -> 5.00 m/s)
int32_t target_speed = 500;
memcpy(&tx_buf[9], &target_speed, sizeof(target_speed));

sendto(udp_socket, tx_buf, len + 1, 0,
       (struct sockaddr*)&stm32_addr, sizeof(stm32_addr));
```